

Tugas 12: Praktikum Mandiri 12 Machine Learning

Hisyam Wildan Alfath - 0110222206

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: hisy22206ti@student.nurulfikri.ac.id

Abstract. Praktikum ini bertujuan untuk menerapkan Principal Component Analysis (PCA) sebagai metode reduksi dimensi pada dataset Breast Cancer dan mengevaluasi pengaruhnya terhadap performa model Support Vector Machine (SVM). Dataset yang terdiri dari 30 fitur numerik direduksi menjadi 3 komponen utama yang mewakili sebagian besar variansi data. Hasil pengujian menunjukkan bahwa model SVM tanpa PCA memiliki akurasi sangat tinggi, namun model SVM dengan PCA tetap mampu mencapai akurasi yang hampir setara meskipun menggunakan fitur yang jauh lebih sedikit. Visualisasi PCA 3D memperlihatkan pemisahan yang cukup jelas antara kelas benign dan malignant, menegaskan bahwa PCA efektif dalam merangkum informasi penting dari data. Secara keseluruhan, PCA terbukti bermanfaat dalam mengurangi kompleksitas data tanpa mengorbankan kinerja model secara signifikan.

1. Praktikum Mandiri 12

- Import Library Dasar

```
[33]
✓ Os
# Import library dasar
import numpy as np
import pandas as pd

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D # untuk 3D plot
# Agar plot tampil rapi
plt.rcParams['figure.figsize'] = (8, 6)
```

Pada tahap ini berbagai library penting diimpor untuk mendukung proses analisis data serta penerapan PCA dan SVM. Library seperti NumPy dan Pandas digunakan untuk pengolahan data numerik dan manipulasi DataFrame, sementara modul dari scikit-learn seperti `train_test_split`, `StandardScaler`, `PCA`, dan `SVC` diperlukan untuk melakukan pembagian data, standarisasi fitur, reduksi dimensi, serta pembangunan model klasifikasi. Selain itu, Matplotlib dan Axes3D digunakan untuk membuat visualisasi data dalam bentuk grafik 2D maupun 3D. Pengaturan

plt.rcParams['figure.figsize'] juga diterapkan agar tampilan plot lebih rapi dan konsisten selama proses analisis berlangsung, memastikan bahwa semua fungsi pemetaan dapat berjalan tanpa error sebelum proses analisis dilanjutkan.

- Membaca Dataset dari Google Drive

```
[34] from google.colab import drive
      drive.mount('/content/drive')
      Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

[35] df = pd.read_csv("/content/drive/MyDrive/Hisyam Wildan Alfath/Semester 7/Machine Learning/Pertemuan 12/Praktikum dan Tugas Mandiri 12/data/data.csv")
      df.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	...	texture_worst
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	...	17.33
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	...	23.41
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	...	25.53
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	...	26.50
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	...	16.67

5 rows x 33 columns

Pada tahap ini Google Drive terlebih dahulu di-mount menggunakan `drive.mount('/content/drive')` sebagai input, sehingga Colab dapat mengakses file yang tersimpan di dalamnya. Selanjutnya, perintah `pd.read_csv("path_dataset")` digunakan sebagai input utama untuk membaca file `data.csv` dan memuatnya ke dalam DataFrame bernama `df`. Output dari proses ini kemudian diverifikasi menggunakan `df.head()`, yang menampilkan beberapa baris pertama dari dataset sebagai output visual, berisi kolom seperti `id`, `diagnosis`, `radius_mean`, `texture_mean`, dan berbagai fitur numerik lainnya. Tampilan awal tersebut menunjukkan bahwa file telah berhasil terbaca dengan benar dan struktur datanya lengkap, sehingga dataset siap digunakan untuk proses preprocessing, PCA, hingga pemodelan menggunakan SVM pada tahap berikutnya.

- Informasi Umum Dataset

```
[36]
✓ Os      # Informasi umum dataset
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
#   Column                                     Non-Null Count  Dtype
---  -
0   id                                         569 non-null    int64
1   diagnosis                                 569 non-null    object
2   radius_mean                             569 non-null    float64
3   texture_mean                             569 non-null    float64
4   perimeter_mean                           569 non-null    float64
5   area_mean                                569 non-null    float64
6   smoothness_mean                           569 non-null    float64
7   compactness_mean                           569 non-null    float64
8   concavity_mean                             569 non-null    float64
9   concave points_mean                       569 non-null    float64
10  symmetry_mean                              569 non-null    float64
11  fractal_dimension_mean                     569 non-null    float64
12  radius_se                                  569 non-null    float64
13  texture_se                                 569 non-null    float64
14  perimeter_se                               569 non-null    float64
15  area_se                                    569 non-null    float64
16  smoothness_se                              569 non-null    float64
17  compactness_se                             569 non-null    float64
18  concavity_se                               569 non-null    float64
19  concave points_se                          569 non-null    float64
20  symmetry_se                                569 non-null    float64
21  fractal_dimension_se                       569 non-null    float64
22  radius_worst                              569 non-null    float64
23  texture_worst                             569 non-null    float64
24  perimeter_worst                           569 non-null    float64
25  area_worst                                569 non-null    float64
26  smoothness_worst                          569 non-null    float64
27  compactness_worst                         569 non-null    float64
28  concavity_worst                           569 non-null    float64
29  concave points_worst                      569 non-null    float64
30  symmetry_worst                            569 non-null    float64
31  fractal_dimension_worst                   569 non-null    float64
32  Unnamed: 32                               0 non-null      float64
dtypes: float64(31), int64(1), object(1)
memory usage: 146.8+ KB
```

Pada tahap ini digunakan perintah `df.info()` sebagai input untuk menampilkan ringkasan struktur dataset yang telah dimuat sebelumnya. Output yang dihasilkan berupa informasi detail mengenai jumlah baris dan kolom, nama setiap fitur, jumlah nilai non-null, serta tipe data masing-masing kolom. Dari hasil tersebut terlihat bahwa dataset memiliki 569 baris dan 33 kolom, dengan beberapa kolom seperti `id` dan `diagnosis` bertipe integer dan object, sementara fitur selebihnya merupakan nilai numerik bertipe float64. Selain itu, juga terlihat bahwa kolom `Unnamed: 32` memiliki nilai kosong (0 non-null), menandakan bahwa kolom tersebut perlu dihapus pada tahap preprocessing. Informasi ini membantu memastikan bahwa struktur data sudah dipahami dengan baik sebelum dilakukan pembersihan, transformasi, dan analisis lebih lanjut menggunakan PCA dan SVM.

- Statistik Deskriptif Dataset

```
[37]
✓ Os # Statistik deskriptif
df.describe()
```

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean	symmetry_mean	...
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	...
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162	...
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	...
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.105000	...
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	...
50%	9.050240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	...
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	...
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	...

8 rows x 32 columns

Pada tahap ini digunakan perintah `df.describe()` sebagai input untuk menghasilkan ringkasan statistik deskriptif dari seluruh fitur numerik dalam dataset. Output yang ditampilkan mencakup informasi seperti jumlah data (`count`), nilai rata-rata (`mean`), standar deviasi (`std`), nilai minimum, kuartil (25%, 50%, dan 75%), serta nilai maksimum untuk setiap kolom numerik. Hasil tersebut memberikan gambaran awal mengenai penyebaran data, misalnya nilai `radius_mean` memiliki rata-rata sekitar 14.1 dengan sebaran standar deviasi 3.52, sedangkan `concavity_mean` memiliki nilai minimum 0 dan maksimum 0.4268. Informasi ini membantu dalam memahami karakteristik data sebelum dilakukan preprocessing lebih lanjut, termasuk pentingnya standardisasi karena skala antar fitur sangat bervariasi.

- Pemeriksaan Missing Value

```
[38]
✓ Os # Cek missing value
df.isnull().sum()
```

	0
id	0
diagnosis	0
radius_mean	0
texture_mean	0
perimeter_mean	0
area_mean	0
smoothness_mean	0
compactness_mean	0
concavity_mean	0
concave points_mean	0
symmetry_mean	0
fractal_dimension_mean	0
radius_se	0
texture_se	0

Pada tahap ini digunakan perintah `df.isnull().sum()` sebagai input untuk memeriksa apakah terdapat nilai kosong (`missing value`) pada setiap kolom dalam

dataset. Output yang dihasilkan berupa daftar kolom beserta jumlah nilai kosongnya, dan berdasarkan hasil yang ditampilkan, seluruh kolom memiliki jumlah missing value sebesar 0, kecuali kolom Unnamed: 32 yang sebelumnya sudah diketahui tidak memiliki data valid. Informasi ini menunjukkan bahwa dataset dalam kondisi bersih dan tidak memerlukan proses imputasi nilai hilang, sehingga dapat langsung dilanjutkan ke tahap preprocessing berikutnya seperti pembersihan kolom tidak penting, encoding label, standardisasi, serta penerapan PCA dan SVM.

- Pembersihan Kolom, Pemeriksaan Duplikasi, dan Pembuatan Label Target

```
[39] ✓ Os df = df.drop(columns=['id', 'Unnamed: 32'], errors='ignore')

[40] ✓ Os # Cek data duplikat
df.duplicated().sum()

np.int64(0)

[41] ✓ Os # Tambah kolom target numerik dari diagnosis
# B = 0 (Benign), M = 1 (Malignant)
df['target'] = df['diagnosis'].map({'B': 0, 'M': 1})

# Distribusi kelas target
print("Nama kelas:", ['Benign (0)', 'Malignant (1)'])
df['target'].value_counts()

Nama kelas: ['Benign (0)', 'Malignant (1)']
count

target
0      357
1      212

dtype: int64
```

Pada tahap ini dilakukan pembersihan dataset dengan menghapus kolom id dan Unnamed: 32 menggunakan perintah `df.drop()`, karena kolom tersebut tidak relevan dalam analisis dan salah satunya berisi nilai kosong. Selanjutnya, `df.duplicated().sum()` digunakan sebagai input untuk memeriksa adanya data duplikat, dan output menunjukkan nilai 0 yang berarti seluruh baris dalam dataset unik. Setelah itu, dibuat kolom target numerik baru melalui perintah `df['diagnosis'].map()` yang mengonversi diagnosis Benign (B) menjadi 0 dan Malignant (M) menjadi 1. Distribusi kelas

kemudian diperoleh dari `df['target'].value_counts()` sebagai output, yang menunjukkan bahwa terdapat 357 data benign dan 212 data malignant. Informasi ini penting untuk memahami komposisi dataset sebelum memasuki tahap preprocessing lanjutan dan pemodelan.

- Pemisahan Fitur dan Label

```
[42]
✓ Os # Fitur dan label
      X = df.drop(columns=['diagnosis', 'target']).values
      y = df['target'].values # label numerik (0 = Benign, 1 = Malignant)

      print("Shape X:", X.shape)
      print("Shape y:", y.shape)

  Shape X: (569, 30)
  Shape y: (569,)
```

Pada tahap ini fitur dan label dipisahkan dengan menghapus kolom diagnosis dan target dari dataset untuk menghasilkan matriks fitur X, sedangkan kolom target dijadikan label y dalam bentuk numerik. Output berupa ukuran X yaitu (569, 30) menunjukkan terdapat 569 sampel dengan 30 fitur, sementara y memiliki 569 nilai label. Pemisahan ini penting sebagai persiapan sebelum data dibagi menjadi train dan test.

- Pembagian Data Train dan Test

```
[43]
✓ Os # Train-test split (80% train, 20% test)
      X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.2, random_state=42, stratify=y
      )

      print("Shape X_train:", X_train.shape)
      print("Shape X_test :", X_test.shape)

  Shape X_train: (455, 30)
  Shape X_test : (114, 30)
```

Pada tahap ini data dibagi menjadi 80% data latih dan 20% data uji menggunakan `train_test_split`, dengan stratifikasi agar proporsi kelas tetap seimbang. Output menunjukkan bahwa data latih berjumlah 455 sampel dan data uji 114 sampel, masing-masing tetap memiliki 30 fitur. Pembagian ini penting untuk memastikan model dilatih dan dievaluasi secara adil.

- Standardisasi Data

```
[44]
✓ Os # Standardisasi (mean=0, std=1)
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train) # fit + transform pada data train
X_test_scaled = scaler.transform(X_test)      # hanya transform pada data test

X_train_scaled[:5]

array([[ 5.18558727e-01,  8.91825791e-01,  4.24631702e-01,
         3.83925436e-01, -9.74743706e-01, -6.89771505e-01,
        -6.88586446e-01, -3.98175254e-01, -1.03915470e+00,
        -8.25056321e-01, -1.09317755e-01, -5.59755400e-02,
        -2.10096206e-01, -1.59132582e-02, -1.00518399e+00,
        -9.11941990e-01, -6.62815884e-01, -6.52561081e-01,
        -7.01889114e-01, -2.75393571e-01,  5.79797697e-01,
         1.31324246e+00,  4.66908134e-01,  4.45982711e-01,
        -5.96154777e-01, -6.34722227e-01, -6.10227299e-01,
        -2.35743918e-01,  5.45663235e-02,  2.18367276e-02],
       [-5.16364088e-01, -1.63971029e+00, -5.41348716e-01,
        -5.42961327e-01,  4.76219058e-01, -6.31833818e-01,
        -6.04281166e-01, -3.03074908e-01,  5.21543093e-01,
        -4.54522896e-01, -6.04377961e-01, -1.00104604e+00,
        -5.85429002e-01, -4.93453793e-01,  4.03212009e-01,
        -7.68173276e-01, -4.79187222e-01,  1.14508478e-01,
        -1.42950761e-01, -5.77397732e-01, -5.82458953e-01,
        -1.69029101e+00, -6.11934288e-01, -5.87013537e-01,
         2.73581959e-01, -8.14844486e-01, -7.12666415e-01,
        -3.23207881e-01, -1.37576237e-01, -9.04401643e-01],
```

Pada tahap ini dilakukan standardisasi menggunakan StandardScaler untuk menyesuaikan skala seluruh fitur agar memiliki mean 0 dan standar deviasi 1. Proses `fit_transform` diterapkan pada data latih untuk menghitung parameter scaling, sedangkan data uji hanya melalui `transform` agar tidak terjadi kebocoran informasi. Output berupa tampilan lima baris pertama dari data yang telah distandardisasi menunjukkan bahwa seluruh nilai fitur kini berada pada skala yang seragam, sehingga lebih optimal ketika digunakan dalam proses PCA dan pelatihan model SVM.

- Model SVM Tanpa PCA (Baseline)

```
[45]
✓ Os # Model SVM tanpa PCA
svm_no_pca = SVC(kernel='rbf', gamma='scale', random_state=42)
svm_no_pca.fit(X_train_scaled, y_train)

# Prediksi dan evaluasi
y_pred_no_pca = svm_no_pca.predict(X_test_scaled)

acc_no_pca = accuracy_score(y_test, y_pred_no_pca)
print("Akurasi SVM tanpa PCA:", acc_no_pca)

print("\nClassification Report (tanpa PCA):")
print(classification_report(y_test, y_pred_no_pca))

Akurasi SVM tanpa PCA: 0.9736842105263158

Classification Report (tanpa PCA):
```

	precision	recall	f1-score	support
0	0.96	1.00	0.98	72
1	1.00	0.93	0.96	42
accuracy			0.97	114
macro avg	0.98	0.96	0.97	114
weighted avg	0.97	0.97	0.97	114

Pada tahap ini model SVM dibangun menggunakan data yang telah distandardisasi tanpa reduksi dimensi sebagai baseline. Model dilatih menggunakan

SVC dengan kernel RBF, kemudian dilakukan prediksi pada data uji untuk menghitung akurasi dan melihat kinerja klasifikasi. Output menunjukkan akurasi sebesar 0.9737, dengan precision, recall, dan f1-score yang tinggi pada kedua kelas, menandakan bahwa model mampu mengklasifikasikan data dengan sangat baik meskipun belum menggunakan PCA.

- Penerapan PCA dengan 3 Komponen Utama

```
[46]
✓ Os
# PCA dengan 3 komponen utama
pca = PCA(n_components=3)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("Shape X_train_pca:", X_train_pca.shape)
print("Shape X_test_pca :", X_test_pca.shape)

Shape X_train_pca: (455, 3)
Shape X_test_pca : (114, 3)
```

Pada tahap ini dilakukan reduksi dimensi menggunakan PCA dengan menentukan tiga komponen utama sebagai representasi baru dari data. Proses `fit_transform` diterapkan pada data latih untuk membentuk komponen PCA, sedangkan data uji hanya ditransformasikan agar tetap konsisten. Output menunjukkan bahwa data yang awalnya memiliki 30 fitur kini direduksi menjadi 3 fitur utama, masing-masing berukuran (455, 3) untuk data latih dan (114, 3) untuk data uji. Reduksi ini membantu menyederhanakan data sekaligus mengurangi kompleksitas model.

- Variansi yang Dijelaskan oleh Komponen PCA

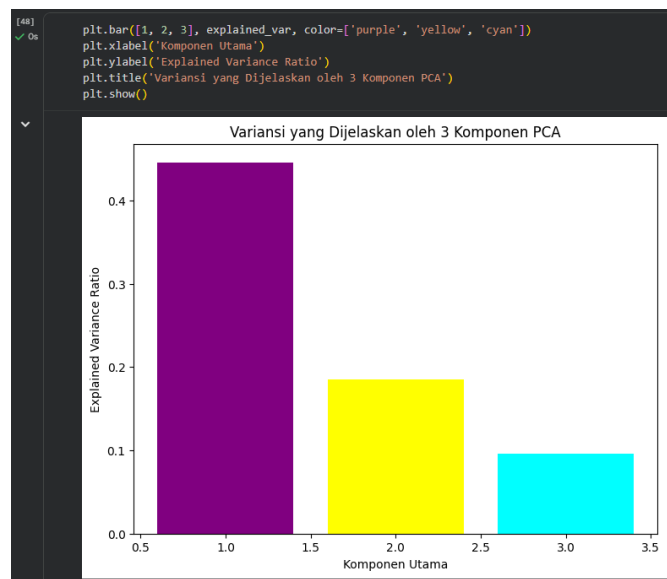
```
[47]
✓ Os
# Variansi yang dijelaskan oleh tiap komponen
explained_var = pca.explained_variance_ratio_

print("Explained Variance Ratio tiap komponen:", explained_var)
print("Total variansi yang dijelaskan 3 komponen pertama:", explained_var.sum())

Explained Variance Ratio tiap komponen: [0.44593522 0.18545255 0.09584641]
Total variansi yang dijelaskan 3 komponen pertama: 0.7272341886241546
```

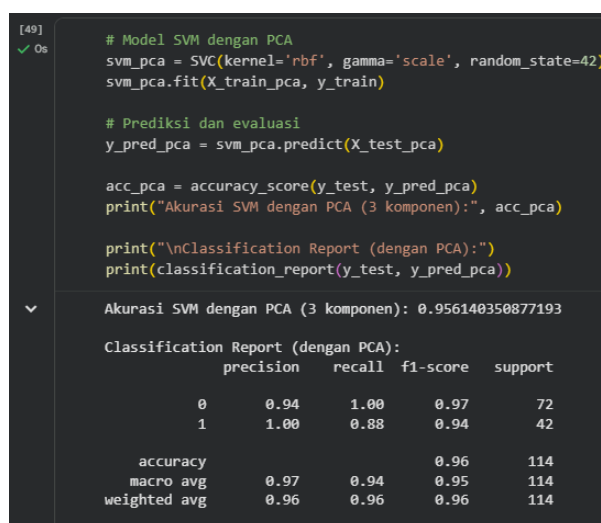
Pada tahap ini digunakan atribut `explained_variance_ratio_` untuk melihat proporsi variansi yang mampu dijelaskan oleh masing-masing komponen PCA. Output menunjukkan bahwa tiga komponen utama menjelaskan sekitar 44.59%, 18.55%, dan 9.58% variansi data. Jika dijumlahkan, ketiga komponen tersebut mampu mempertahankan sekitar 72.72% informasi dari data asli, yang menunjukkan bahwa PCA berhasil mereduksi dimensi tanpa kehilangan terlalu banyak informasi penting.

- Visualisasi Variansi Komponen PCA



Grafik batang ini menunjukkan seberapa besar variansi yang dapat dijelaskan oleh masing-masing dari tiga komponen utama PCA. Komponen pertama memiliki kontribusi variansi paling besar diikuti komponen kedua dan ketiga, yang divisualisasikan menggunakan warna berbeda (ungu, kuning, dan cyan). Visualisasi ini membantu memahami seberapa efektif reduksi dimensi yang dilakukan, sekaligus memastikan bahwa sebagian besar informasi penting tetap dipertahankan oleh komponen-komponen tersebut.

- Model SVM Setelah Reduksi Dimensi dengan PCA



Model SVM kemudian dilatih menggunakan data yang telah direduksi menjadi tiga komponen utama oleh PCA. Setelah dilakukan prediksi pada data uji, diperoleh

akurasi sebesar 0.9561, sedikit lebih rendah dibanding model tanpa PCA namun tetap menunjukkan performa yang sangat baik. Hasil classification report memperlihatkan nilai precision, recall, dan f1-score yang tinggi pada kedua kelas, menandakan bahwa model masih mampu mengenali pola data dengan baik meskipun jumlah fitur telah diperkecil secara signifikan.

- Visualisasi PCA dalam Ruang 3 Dimensi

```
[50]
✓ Os
# Visualisasi 3D PCA (menggunakan data train)
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection='3d')

scatter = ax.scatter(
    X_train_pca[:, 0],
    X_train_pca[:, 1],
    X_train_pca[:, 2],
    c=y_train,
    s=60
)

ax.set_title('Visualisasi PCA (3 Komponen) - Dataset Breast Cancer')
ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.set_zlabel('PC3')

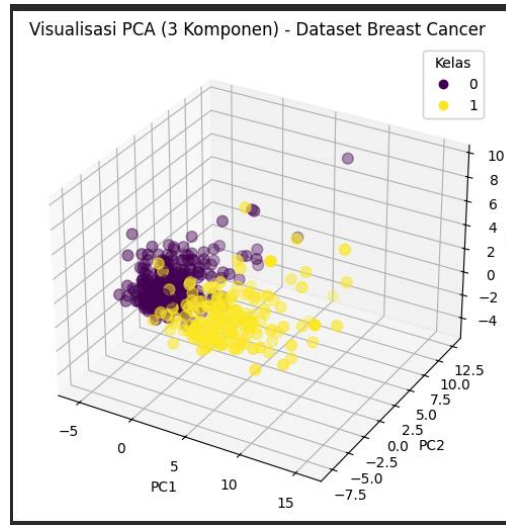
# Tambahkan legend berdasarkan kelas
legend1 = ax.legend(
    *scatter.legend_elements(),
    title="Kelas"
)

ax.add_artist(legend1)

plt.show()
```

Visualisasi ini menampilkan data Breast Cancer yang telah direduksi menjadi tiga komponen utama PCA dalam bentuk scatter plot 3D. Setiap titik merepresentasikan satu sampel, sementara pewarnaan didasarkan pada label kelas sehingga pola pemisahan antara benign dan malignant dapat terlihat lebih jelas. Plot ini membantu memahami bagaimana PCA merangkum struktur data ke dalam ruang berdimensi lebih rendah, sekaligus menunjukkan bahwa kedua kelas memiliki kecenderungan berada pada area yang berbeda dalam ruang PCA.

- Output Visualisasi PCA 3D



Gambar tersebut menunjukkan sebaran data Breast Cancer yang telah direduksi menjadi tiga komponen PCA dan divisualisasikan dalam ruang tiga dimensi. Titik berwarna ungu mewakili kelas Benign (0), sedangkan titik berwarna kuning mewakili kelas Malignant (1). Terlihat bahwa kedua kelas membentuk kelompok yang cenderung terpisah, terutama pada sumbu PC1 dan PC2, menunjukkan bahwa PCA mampu menangkap pola yang membedakan kedua kelas. Plot ini membantu memperlihatkan bagaimana struktur data dapat diringkas menjadi tiga dimensi tanpa kehilangan karakteristik utama yang relevan untuk klasifikasi.

- Perbandingan Model Tanpa PCA dan Dengan PCA

```
[51]
✓ Os
# Membandingkan hasil model tanpa dan dengan PCA
comparison = pd.DataFrame({
    'Model': ['SVM Tanpa PCA', 'SVM Dengan PCA (3 Komponen)'],
    'Jumlah Fitur': [X_train_scaled.shape[1], X_train_pca.shape[1]],
    'Akurasi': [acc_no_pca, acc_pca],
    'Variansi Total PCA': [None, explained_var.sum()]
})

comparison
```

	Model	Jumlah Fitur	Akurasi	Variansi Total PCA
0	SVM Tanpa PCA	30	0.973684	NaN
1	SVM Dengan PCA (3 Komponen)	3	0.956140	0.727234

Tabel perbandingan ini menunjukkan perbedaan kinerja antara model SVM yang menggunakan seluruh 30 fitur dan model SVM yang dilatih setelah reduksi dimensi menjadi 3 komponen PCA. Model tanpa PCA menghasilkan akurasi sebesar

0.9737, sedangkan model dengan PCA memperoleh akurasi 0.9561, yang hanya sedikit lebih rendah meski jumlah fitur berkurang drastis. Nilai variansi total PCA sebesar 0.7272 menandakan bahwa tiga komponen utama mampu mempertahankan sekitar 72% informasi asli. Hasil ini membuktikan bahwa PCA dapat menyederhanakan data secara signifikan tanpa mengorbankan performa model secara berarti.

- Visualisasi Perbandingan Akurasi Model SVM



Diagram batang ini membandingkan akurasi model SVM tanpa PCA dan model SVM yang menggunakan tiga komponen PCA. Warna ungu merepresentasikan model tanpa PCA, sedangkan kuning menunjukkan model dengan PCA, selaras dengan warna visualisasi sebelumnya. Hasil grafik memperlihatkan bahwa kedua model memiliki akurasi tinggi dan perbedaannya sangat tipis, yaitu sekitar 0.97 untuk model tanpa PCA dan 0.96 untuk model dengan PCA. Visualisasi ini mempertegas bahwa reduksi dimensi menggunakan PCA mampu menyederhanakan data secara signifikan tanpa mengurangi performa model secara drastis.

- Kesimpulan

Melalui praktikum mandiri ini dapat disimpulkan bahwa penerapan PCA berhasil mereduksi jumlah fitur dari 30 menjadi 3 komponen utama sambil tetap mempertahankan sebagian besar informasi penting dari dataset Breast Cancer. Model SVM yang dilatih tanpa PCA menunjukkan akurasi sangat tinggi, namun model dengan PCA juga mampu memberikan performa yang hampir setara meskipun menggunakan fitur yang jauh lebih sedikit. Hal ini menunjukkan bahwa PCA efektif dalam menyederhanakan data dan mengurangi kompleksitas model tanpa menurunkan akurasi secara signifikan. Selain itu, visualisasi PCA 3D memperlihatkan pemisahan yang cukup jelas antara kelas benign dan malignant, mendukung bahwa komponen utama yang dihasilkan mampu merepresentasikan struktur data dengan baik.

Link Praktikum 12:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%2012/Praktikum%20dan%20Tugas%20Mandiri%2012/notebooks/praktikum12.ipynb

Tugas Mandiri 12:

https://github.com/HisyamWildan/TI03_HisyamW.A_0110222206/blob/main/Pertemuan%2012/Praktikum%20dan%20Tugas%20Mandiri%2012/notebooks/latihan12.ipynb